

Preemptive scheduling with finite capacity input buffers

Nicholas G. Hall

*Department of Management Sciences, The Ohio State University,
Columbus, OH 43210-1271, USA*

Marc E. Posner

*Industrial Engineering, The Ohio State University,
Columbus, OH 43210-1271, USA*

Chris N. Potts

*Faculty of Mathematical Studies, University of Southampton,
Southampton SO17 1BJ, UK*

In many scheduling problems, an arriving job is stored in an input buffer until it starts to be processed. Also, it may be necessary to hold a partially completed preempted job in an input buffer until processing of this job resumes. In the scheduling literature, most problems have been studied using the implicit assumption that the buffer has infinite capacity. We study preemptive single machine scheduling problems where the buffer capacity is finite. In this scheduling environment, jobs may be lost either because of insufficient input buffer capacity, or because due date requirements cannot be met. We examine problems where the objective is to minimize the weighted or unweighted number of lost jobs. Various assumptions about the generality of the data are examined. We present a complexity classification for various problems, either by deriving an efficient algorithm, or by proving that such an algorithm is unlikely to exist.

Keywords: Preemptive single machine scheduling, finite input buffers, polynomial time algorithm, NP-complete.

1. Introduction

We consider a class of deterministic single machine scheduling problems where there are *input buffers*. The jobs have varying release dates and preemption is allowed. When a job is released, it may either be processed immediately by the machine, discarded, or stored in an input buffer for future processing. If a job is preempted, it must return to the buffer for storage until it resumes being processed. Although we make various assumptions about the relationship of the buffer capacity to the number of jobs, the capacity of the input buffer is finite. This is contrary to the usual assumption in the scheduling literature that the buffer capacity is infinite.

Scheduling problems with finite capacity input buffers arise in many types of manufacturing environments. For example, limited warehouse space for raw materials can be modeled by finite capacity input buffers. In modern manufacturing systems, the provision of buffer space (for input buffers, intermediate buffers between

machines, and output buffers for the storage of finished goods) is expensive. Holding products in a buffer incurs costs such as insurance, record-keeping and deterioration. Also, the opportunity cost of financing the inventory is often considerable (McClain and Thomas [6]). This suggests restricting the buffer space as a means to reduce costs. Consequently, it is important to consider scheduling problems where there is limited buffer space.

We study preemptive scheduling problems where the objective is to minimize the total weight (or value) associated with lost jobs. The assumption of job preemption is relevant when the setup times and costs are insignificant, and when the processing of jobs can be restarted after an interruption without the need to repeat earlier work.

Jobs may be lost because of finite input buffer capacity, or because of due date restrictions. Thus, after a job is discarded, it uses neither the machine nor the buffer. There are strong parallels between lost jobs models and classical late jobs models. In both cases, a cost is incurred if the job does not complete on time. In the late jobs model, there is an assumption that the machine has enough capacity to process all jobs and that there is infinite buffer space. The late jobs are processed after all of the on-time jobs are completed. We could similarly assume, in this work, that the machine has enough capacity to process all jobs and that discarded jobs are placed in a remote location with infinite capacity that is difficult to access. Then, we could retrieve and process the discarded jobs after all of the on-time jobs are completed. These models and the models we propose are identical with respect to the set of feasible schedules and the cost of the schedules.

We consider various assumptions about the system to be scheduled. These include:

- (1) Weights of jobs: arbitrary; unit.
- (2) Buffer capacity: arbitrary; constant.
- (3) Due dates: arbitrary; infinite.
- (4) Storage requirements of jobs: arbitrary; unit.

The storage requirement of a job is the space that it occupies in the buffer. For most problems, we provide either an efficient algorithm or a proof of NP-hardness.

Nawijn [7] studies a nonpreemptive, finite capacity input buffer problem in which the jobs that are not lost are processed in their order of arrival. Also, Nawijn et al. [8] derive a polynomial time algorithm for the nonpreemptive problem of minimizing the number of lost jobs when the buffer capacity is a positive constant. They prove that the problem of minimizing the weighted number of lost jobs is binary NP-hard. Hall et al. [4] provide a computational complexity classification for several nonpreemptive single machine scheduling problems with finite capacity input buffers.

In some situations, completed jobs can be held in output buffers until a customer is ready to accept delivery. Hall et al. [3] present a computational complexity classification of various single machine scheduling problems with finite capacity output buffers.

The paper is organized as follows. In section 2, notation and formal definitions of the scheduling problems are presented. Also, we summarize our algorithmic and computational complexity results and provide some preliminary analysis. Preemptive problems with the objective of minimizing the number of lost jobs are analyzed in section 3, while corresponding analysis for minimizing the weighted number of lost jobs is given in section 4. Some concluding remarks are contained in section 5.

2. Preliminaries

2.1. Notation and classification

Let $N = \{1, \dots, n\}$ denote the set of jobs. Also, let w_j denote the weight (or value), r_j the release date, d_j the due date, and p_j the processing time of job j , for $j \in N$. Let $r_{\min} = \min_{j \in N} \{r_j\}$ and $p_{\max} = \max_{j \in N} \{p_j\}$. The buffer storage requirement of job j is denoted by s_j , and the buffer capacity is denoted by b_I . We assume that w_j , r_j , d_j , p_j , and s_j are integer, and that $r_j + p_j \leq d_j$, for all $j \in N$. Let S be an n -dimensional column vector with elements $s_1, \dots, s_n$, respectively.

The convention is used that the *scheduling* of a job defines a decision as to when the *processing* of that job takes place. We refer to a job j as being *available* at time t if either $r_j = t$ or j is in the buffer at time t . Because there is no penalty associated with preemption, we restrict our analysis to schedules where the machine is only idle when no job which completes by its due date is available for processing. A schedule with this property is called an *active* schedule.

Let σ be a schedule, which may be preemptive and only contains jobs that complete processing no later than their due dates. We define

$$U_j(\sigma) = \begin{cases} 1 & \text{if job } j \text{ does not appear in } \sigma, \\ 0 & \text{otherwise.} \end{cases}$$

When there is no ambiguity, we simplify $U_j(\sigma)$ to U_j .

The standard classification scheme for scheduling problems (Graham et al. [2]) is $\psi_1 | \psi_2 | \psi_3$, where ψ_1 indicates the scheduling environment, ψ_2 describes the job characteristics or restrictive requirements, and ψ_3 defines the objective function to be minimized. This scheme is extended by Hall et al. [4] to provide for finite capacity input buffers. Under ψ_2 , we have

$b_I = b$: input buffer capacity is a positive constant, b ;

b_I : input buffer capacity is arbitrary;

$pmtn$: the preemption of jobs is allowed;

r_j : job j becomes available for processing at its release date, r_j ;

$d_j = \infty$: all due dates are infinite;

$s_j = 1$: all jobs have a unit buffer storage requirement.

The objective functions that we consider under ψ_3 are

$\sum U_j$ = the number of jobs lost;

$\sum w_j U_j$ = the total weight of jobs lost.

An example of the classification scheme is problem $1|b_I, pmtn, r_j, s_j = 1|\sum U_j$. This denotes a single machine, preemptive scheduling problem with release dates, where an input buffer of arbitrary given capacity is available to hold arriving jobs, and each job has a unit storage requirement. The objective of this problem is to minimize the number of lost jobs.

2.2. Overview of the results

Table 1 presents our results for preemptive scheduling problems. A “Yes” in the $d_j = \infty$ column implies that all due dates are arbitrarily large so that jobs are lost only because of the input buffer constraint, “No” implies that there are no specified restrictions on the due dates, and “Any” implies either Yes or No. In the $s_j = 1$ column, the “Yes”, “No” and “Any” are similarly defined. We use Unary and Binary *NPC* to indicate that the equivalent recognition version of a problem is NP-complete with respect to a unary and binary encoding of the data, respectively. Related definitions are given by Garey and Johnson [1]. Included in the appropriate cell is a reference to where a proof of that result can be found.

Table 1
The complexity of the problems.

Objective	Buffer type	$d_j = \infty$	$s_j = 1$	Complexity results	
$\sum U_j$	$b_I = b$	Yes	Yes	$O(n \log n)$	Thm 3.1
			No	Open, $O(n^{b+3} p_{\max}^{b+1})$	Cor 3.1
		No	Any	Open, $O(n^{b+3} p_{\max}^{b+1})$	Thm 3.2, Cor 3.1
	b_I	Any	No	Unary <i>NPC</i>	Thm 3.3, Cor 3.2
		Yes	Yes	$O(n \log n)$	Thm 3.1
		No	Yes	Open	
$\sum w_j U_j$	$b_I = b$	Any	Any	Binary <i>NPC</i> $O(n^{b+3} p_{\max}^{b+2})$	Thm 4.2, Cor 4.2 Thm 4.1, Cor 4.1
	b_I	Any	Yes	Binary <i>NPC</i> , Open	Cor 4.3
			No	Unary <i>NPC</i>	Cor 4.4

2.3. Preemption, buffers and scheduling times

We begin with a result that permits us to restrict the times when preemptions occur.

Lemma 2.1 In any input buffer scheduling problem, there exists an optimal schedule in which preemption occurs only at release dates.

Proof Suppose that, in some optimal schedule, t is the first time that there is a preemption, but no job is released at that time. Let job i be the job that is preempted at time t . Let $t - t_i$ denote the earliest time for which job i is processed throughout $[t - t_i, t]$, and no job k has a release date satisfying $t - t_i < r_k < t$. Also, let $t + t_j$ denote the latest time for which some job j with $r_j < t$ is processed throughout $[t, t + t_j]$. Suppose that job i completes before job j and has p'_i units of processing scheduled after time t . Then, we can interchange the last $\min\{p'_i, t_j\}$ units of processing of job i with the first $\min\{p'_i, t_j\}$ units of job j that are scheduled in $[t, t + t_j]$ without affecting the feasibility of the schedule. Moreover, the completion time of job i is reduced. We now have a schedule where the cost has not increased and any preemption before time $t + t_j$ occurs at a release date. Similarly, if job j completes before job i and has p'_j units of processing scheduled after time t , then we can interchange the last $\min\{p'_j, t_i\}$ units of processing of job j with the first $\min\{p'_j, t_i\}$ units of job i that are scheduled in $[t - t_i, t]$ without affecting feasibility or increasing cost. Thus, we again obtain a schedule in which any preemption before time $t + t_j$ occurs at a release date. These transformations can be repeated until all preemptions that do not occur at release dates are removed. $\square$

Since all data are integer, lemma 2.1 allows us to restrict our search to schedules in which, throughout each integer unit time interval, the machine is either idle or processing a single job. Further, if $p_j = 1$ for $j \in N$, then preemption does not reduce the cost of an optimal schedule. Even when processing times are arbitrary, if $r_j = 0$ for $j \in N$, then we can restrict our attention to nonpreemptive schedules. Also, if the buffer capacity is zero, then preemption is not possible due to the unavailability of storage space. Since this work is concerned with preemptive scheduling, we only consider problems with arbitrary processing times and arbitrary release dates, and where the buffer capacity is a positive constant or is arbitrary, i.e., specified as problem input.

A specification of the buffer and machine contents is sometimes needed. Let $B = (B_1, \dots, B_n)$ and $R = (R_1, \dots, R_n)$ be n -dimensional row vectors where

$$B_i = \begin{cases} 1 & \text{if job } i \text{ is in the buffer or on the machine,} \\ 0 & \text{otherwise;} \end{cases}$$

$$R_i = \begin{cases} \text{remaining (integer) processing time of job } i & \text{if } B_i = 1, \\ 0 & \text{otherwise.} \end{cases}$$

Let $(\mathcal{B}, \mathcal{R})$ be the set of all $2n$ -dimensional row vectors that correspond to sets of jobs that can occupy the machine and buffer, and their associated amounts of remaining processing time. More formally, $(B, R) \in (\mathcal{B}, \mathcal{R})$ if $\sum_{i=1}^n s_i B_i - \max_{i \in N} \{s_i B_i\} \leq b_I$, and

$R_i \in \{1, \dots, p_i\}$ when $B_i = 1$. Also, let $\mathbf{0}$ be an n -dimensional row vector of 0's, and e_i be an n -dimensional row vector of 0's except that the i th element is 1.

We also need to specify the time intervals within which processing can occur. Let $\mathcal{T}$ contain all potential integer times t for which a job can be processed in the interval $[t-1, t]$ in some feasible active schedule. Our computation of $\mathcal{T}$ ignores buffer capacity constraints and due date considerations. Consequently, it is possible that some intervals in $\mathcal{T}$ would not be used for processing in any feasible schedule. To compute $\mathcal{T}$, we construct an active smallest release date (SRD) schedule by sequencing the available jobs nonpreemptively in nondecreasing order of their release dates. Then, we let $\mathcal{T} = \{t \mid \text{some job is scheduled in the interval } [t-1, t] \text{ in an SRD schedule and } t \text{ is integer}\}$.

The following result establishes an upper bound on $|\mathcal{T}|$.

Lemma 2.2 For any active schedule in an input buffer scheduling problem, each job is processed in an interval $[t-1, t]$ where $t \in \mathcal{T}$. Moreover, $|\mathcal{T}| \leq np_{\max}$.

Proof Suppose that, in an active schedule σ , some job is processed in the interval $[t'-1, t']$, where $t' \notin \mathcal{T}$. Further, suppose that interval $[t-1, t]$ is the last unit period of machine idle time before t' . Since σ is an active schedule, the machine is not idle during $[t-1, t]$ if a job is in the buffer at this time. Consequently, $r_j \geq t$ for each job j that is processed in $[t, t']$. Because σ has no idle time in $[t, t']$ and all jobs processed in this interval have release dates that are no smaller than t , an SRD schedule also processes jobs throughout the interval $[t, t']$. This contradicts the assumption that $t' \notin \mathcal{T}$.

From the construction of $\mathcal{T}$, we have $|\mathcal{T}| = \sum_{j=1}^n p_j \leq np_{\max}$. □

3. Number of lost jobs

We begin our analysis of preemptive scheduling problems by considering the minimization of $\sum_{j=1}^n U_j$. First, we consider problem $1|b_I, pmtn, r_j, d_j = \infty, s_j = 1|\sum U_j$. For this problem, we present Algorithm SRPT, which uses the shortest remaining processing time to decide which job to process next and which jobs to store in the buffer. Intuitively, by processing short jobs and discarding those jobs with the longest remaining processing times when the buffer capacity constraint forces some jobs to be lost, future demands on buffer space are minimized. Decisions about which jobs to discard are made at release dates. Also, when the completion of a job does not correspond to a release date, a job with the shortest remaining processing time is removed from the buffer (unless it is empty) and processed. Algorithm SRPT is also applicable for problem $1|b_I = b, pmtn, r_j, d_j = \infty, s_j = 1|\sum U_j$.

Algorithm SRPT

Input: r_i, p_i for $i \in N$, and b_I .

Step 1 (*Initialization*).

Index the jobs such that $r_1 \leq \dots \leq r_n$.

Set $\sum U_j \leftarrow 0$, $k \leftarrow 1$, $t \leftarrow r_1$, $B_i \leftarrow 0$ and $R_i \leftarrow 0$ for $i \in N$, and $r_{n+1} \leftarrow \infty$.

Step 2 (*Discard jobs*).

Set $l \leftarrow \max\{i \mid i \in N, r_i = r_k\}$. Set $B_i \leftarrow 1$ and $R_i \leftarrow p_i$ for $i = k, \dots, l$, and $k \leftarrow l + 1$.

If $\sum_{i=1}^n B_i > b_I + 1$, then let F be a set of $\sum_{i=1}^n B_i - b_I - 1$ jobs with the largest R_i values. Set $B_i \leftarrow 0$ and $R_i \leftarrow 0$ for $i \in F$ and $\sum U_j \leftarrow \sum U_j + |F|$.

Step 3 (*Schedule job at release date*).

Set $j \leftarrow \operatorname{argmin}\{R_i \mid i \in N, R_i > 0\}$ and $t' \leftarrow \min\{r_k, t + R_j\}$.

Process job j in the interval $[t, t']$.

Set $R_j \leftarrow R_j - t' + t$ and $t \leftarrow t'$. If $R_j = 0$, then set $B_j \leftarrow 0$.

Step 4 (*Scheduling next job test*).

If $r_k = t$, then go to step 2.

If $B_i = 0$ for all $i \in N$, and $k \leq n$, then set $t \leftarrow r_k$ and go to step 2.

If $B_i = 0$ for all $i \in N$, and $k = n + 1$, then terminate.

Step 5 (*Schedule job at completion time*).

Set $j \leftarrow \operatorname{argmin}\{R_i \mid i \in N, R_i > 0\}$ and $t' \leftarrow \min\{r_k, t + R_j\}$.

Process job j in the interval $[t, t']$.

Set $R_j \leftarrow R_j - t' + t$ and $t \leftarrow t'$. If $R_j = 0$, then set $B_j \leftarrow 0$.

Go to step 4.

Note that the schedule generated by Algorithm SRPT may have jobs which are only partially processed. This processing can be removed from the schedule on termination of the algorithm in $O(n)$ time.

Theorem 3.1 Algorithm SRPT generates an optimal schedule for problems $1 \mid b_I = b, \text{pmtn}, r_j, d_j = \infty, s_j = 1 \mid \sum U_j$ and $1 \mid b_I, \text{pmtn}, r_j, d_j = \infty, s_j = 1 \mid \sum U_j$ in $O(n \log n)$ time.

Proof We first justify the use of the shortest remaining processing time rule in Algorithm SRPT to schedule jobs that are not lost. Suppose that there is an optimal schedule where both jobs i and j are completed, and for some time t , $r_i \leq t$, $r_j \leq t$, job j is processed in the interval $[t, t + 1]$, and the remaining processing time for job i at time t is less than that for job j . We transform the schedule by interchanging units of jobs i and j processed after time t so that all units of i precede those of j processed after time t . Observe that i completes in the transformed schedule before j completes in the original schedule. Since $d_j = \infty$ and $s_i = s_j = 1$, the transformed schedule is feasible. Because it has the same cost, the transformed schedule is optimal. Repetition

of this argument shows that scheduling jobs, which are not lost by the shortest remaining processing time rule, yields an optimal schedule.

Now, we justify the choice of the jobs that are discarded in step 3. From lemma 2.1, we can restrict our attention to schedules where preemptions occur only at release dates. Given any schedule, it is possible to identify a minimum cardinality subset of jobs that are discarded at a release date because of the buffer capacity constraint. Suppose that the jobs processed in the SRPT schedule are different from those processed in some optimal schedule. Consider the first release date where some job i is discarded by SRPT, while an alternative job l is discarded in the optimal schedule. From step 3 of SRPT, the shortest remaining processing time of job i is no smaller than that of job l . Now, interchange jobs i and l in the optimal schedule. Since this interchange does not increase the total number of lost jobs, nor is the buffer capacity constraint violated, another optimal schedule results. We can repeat this argument until the optimal schedule is transformed to the schedule generated by SRPT.

We now establish the time complexity of SRPT. Step 1 requires $O(n \log n)$ time to index the jobs. To implement steps 2 through 5 efficiently, the jobs for which $B_i = 1$ are stored in a heap, with the remaining processing time of a job as the key. The heap requires $O(n \log n)$ time to insert or delete a job (Kruse [5]). Since jobs can only be preempted at release dates, the computations associated with updating k , t and $\sum U_j$ require $O(n)$ time. Steps 2 through 5 are executed at most n times. Thus, the overall time complexity of Algorithm SRPT is $O(n \log n)$. $\square$

To solve problem $1|b_I = b, pmt_n, r_j|\sum U_j$, we propose a pseudo-polynomial time dynamic programming algorithm. This algorithm generates (partial) schedules each of which corresponds to some state (V, B, R) , where V is the number of jobs that have been completed in the schedule, B is a vector that indicates which jobs are either in the buffer or on the machine at the end of the schedule (any jobs that are released at the end of the schedule and not lost are included in B), and R is a vector that specifies the remaining processing times for the jobs of B . Let $f(V, B, R)$ denote the minimum completion time among all schedules that achieve the state (V, B, R) .

The state (V, B, R) is reached from some previous state $(\bar{V}, \bar{B}, \bar{R})$ by scheduling either one unit of processing time for some job j or a period of idle time. In the former case, if the final unit of processing for job j is scheduled, then $\bar{V} = V - 1$; otherwise, $\bar{V} = V$. The relationship between $\bar{B}$ and B depends on whether the final unit of processing for job j is being scheduled. Suppose that the schedule corresponding to (V, B, R) completes at the release date of the last job to enter the buffer or machine, which is $t = \max\{r_i | i \in N, B_i = 1\}$. To specify the jobs which are released at time t and are not lost, we define, for $i \in N$, $B'_i = 1$ if $B_i = 1$ and $r_i = t$; $B'_i = 0$ otherwise. Then, $\bar{B}_i = B_i - B'_i$ for each job i that does not complete processing at time t . If $\bar{B} = \mathbf{0}$, then a period of idle time up to the release date of some job is scheduled.

The recurrence relation is

$$f(V, B, R) = \begin{cases} t & \text{if } t = 1 + \min_{j \in H_1} \{f(V-1, B+e_j - B', R+e_j - \sum_{i=1}^n B'_i p_i e_i)\} \quad (1) \\ & \text{or } t = 1 + \min_{j \in H_2} \{f(V, B-B', R+e_j - \sum_{i=1}^n B'_i p_i e_i)\} \quad (2) \\ & \text{or } t \geq 1 + f(V, \mathbf{0}, \mathbf{0}), B = B' \text{ and } R = \sum_{i=1}^n B'_i p_i e_i, \quad (3) \\ 1 + \min \begin{cases} \min_{j \in H_3} \{f(V-1, B+e_j, R+e_j)\} \\ \min_{j \in H_4} \{f(V, B, R+e_j)\} \end{cases} & \text{otherwise,} \quad (4) \end{cases} \quad (5)$$

for $(B, R) \in (\mathcal{B}, \mathcal{R})$, and $V = 1, \dots, n$,

where $t = \max\{r_i \mid i \in N, B_i = 1\}$, $B' = (B'_1, \dots, B'_n)$ with

$$B'_i = \begin{cases} 1 & \text{if } B_i = 1 \text{ and } r_i = t, \\ 0 & \text{otherwise,} \end{cases}$$

and where

$$\begin{aligned} H_1 &= \{k \mid k \in N, R_k = 0, f(V-1, B+e_k - B', R+e_k - \sum_{i=1}^n B'_i p_i e_i) \\ &\quad \leq d_k - 1, (B - B')S \leq b\}, \\ H_2 &= \{k \mid k \in N, R_k > 0, B'_k = 0, f(V, B - B', R+e_k - \sum_{i=1}^n B'_i p_i e_i) \\ &\quad \leq d_k - R_k - 1, (B - e_k - B')S \leq b\}, \\ H_3 &= \{k \mid k \in N, R_k = 0, f(V-1, B+e_k, R+e_k) \leq d_k - 1, BS \leq b\} \text{ and} \\ H_4 &= \{k \mid k \in N, R_k > 0, f(V, B, R+e_k) \leq d_k - R_k - 1, (B - e_k)S \leq b\}. \end{aligned}$$

The boundary condition is

$$\begin{aligned} f(0, B, \sum_{i=1}^n B_i p_i e_i) &= r_{\min} \text{ for all } B \in \mathcal{B} \\ \text{such that } \{i \mid i \in N, B_i = 1\} &\subseteq \{i \mid i \in N, r_i = r_{\min}\}. \end{aligned}$$

By implication, all other initial values are set to infinity.

The optimal value is given by

$$\max\{V \mid f(V, \mathbf{0}, \mathbf{0}) < \infty\}.$$

We refer to this dynamic programming approach as Algorithm DPV (where the V describes the first state variable).

Theorem 3.2 Algorithm DPV generates an optimal schedule for problem $1 \mid b_I = b, p_{\min}, r_j \mid \sum U_j$ in $O(n^{b+3} p_{\max}^{b+1})$ time.

Proof Equations (1), (2) and (3) consider the situation where at least one job of $\{i \mid i \in N, B_i = 1\}$ is released at time $f(V, B, R)$. In equation (1), the last unit of processing of job j completes at $f(V, B, R)$. Equation (2) considers the case where one unit of

job j is processed, but job j does not complete. Equation (3) is used to schedule a period of idle time up to some subsequent release date. As observed in section 2.1, we only allow this to occur when the buffer is empty. Equations (4) and (5) are similar to (1) and (2), respectively, except that $f(V, B, R)$ is not the release date of any job in $\{i | i \in N, B_i = 1\}$. Since all possible ways of reaching state (V, B, R) are considered, Algorithm DPV generates an optimal schedule.

We now examine the time complexity of Algorithm DPV. Since the number of jobs in a buffer-machine configuration is no larger than $b + 1$, $|(B, R)|$ is $O(n^{b+1} p_{\max}^{b+1})$. Because $V \leq n$, the number of possible states (V, B, R) is $O(n^{b+2} p_{\max}^{b+1})$. Each time the recurrence relation is used, $O(n)$ comparisons are required, each of which takes constant time. Thus, the time complexity of the recurrence relation is $O(n^{b+3} p_{\max}^{b+1})$. The computation of the optimal schedule from the final states requires $O(n)$ time. Thus, the overall time complexity of Algorithm DPV is $O(n^{b+3} p_{\max}^{b+1})$. $\square$

Corollary 3.1 Algorithm DPV generates an optimal schedule for problems $1|b_I = b, pmtn, r_j, d_j = \infty | \sum U_j$ and $1|b_I = b, pmtn, r_j, s_j = 1 | \sum U_j$ in $O(n^{b+3} p_{\max}^{b+1})$ time.

Proof The result follows from theorem 3.2, since these problems are special cases of $1|b_I = b, pmtn, r_j | \sum U_j$. $\square$

Problems $1|b_I = b, pmtn, r_j, d_j = \infty | \sum U_j$, $1|b_I = b, pmtn, r_j, s_j = 1 | \sum U_j$ and $1|b_I = b, pmtn, r_j | \sum U_j$ remain open with respect to polynomial time solvability.

We now consider some additional problems where buffer capacity is arbitrary. In our analysis, we need the following problem which is known to be unary NP-complete.

3-Partition (Garey and Johnson [1]): given $3m$ elements with integer sizes $a_1, \dots, a_{3m}$, where $\sum_{i=1}^{3m} a_i = my$, and $y/4 < a_i < y/2$ for $i = 1, \dots, 3m$, does there exist a partition $S_1, \dots, S_m$ of the index set $\{1, \dots, 3m\}$ such that $|S_j| = 3$ and $\sum_{i \in S_j} a_i = y$, for $j = 1, \dots, m$?

Theorem 3.3 The recognition version of problem $1|b_I, pmtn, r_j, d_j = \infty | \sum U_j$ is unary NP-complete.

Proof We use a reduction from 3-Partition. Consider the following instance of $1|b_I, pmtn, r_j, d_j = \infty | \sum U_j$:

$$\begin{aligned} n &= 5m, \\ r_j &= 0, & j &= 1, \dots, 3m, \\ p_j &= a_j, & j &= 1, \dots, 3m, \\ s_j &= a_j, & j &= 1, \dots, 3m, \\ r_j &= ly + 2(l-1), & k &= 1, 2, l = 1, \dots, m, j = 3m + 2(l-1) + k, \end{aligned}$$

$$\begin{aligned}
p_j &= 1, & k &= 1, 2, l = 1, \dots, m, j = 3m + 2(l - 1) + k, \\
s_j &= ly, & k &= 1, 2, l = 1, \dots, m, j = 3m + 2(l - 1) + k, \\
b_I &= my.
\end{aligned}$$

Let $J_1 = \{1, \dots, 3m\}$ and $J_2 = \{3m + 1, \dots, 5m\}$. The jobs of J_1 are created from the 3-Partition data and are all released at time zero. The jobs of J_2 can be partitioned into m blocks, each containing two identical jobs.

We prove that there exists a schedule for this instance of $1|b_I, pmtn, r_j, d_j = \infty|\sum U_j$ with $\sum U_j \leq 0$ if and only if there exists a solution to 3-Partition. We assume without loss of generality that, if there does exist a 3-Partition, then the elements are numbered such that $a_{3i-2} + a_{3i-1} + a_{3i} = y$ for $i = 1, \dots, m$.

($\Rightarrow$) Consider the nonpreemptive schedule

$$\sigma = (1, 2, 3, 3m + 1, 3m + 2, 4, 5, 6, 3m + 3, 3m + 4, \dots, 3m - 2, 3m - 1, 3m, 5m - 1, 5m).$$

At time zero, all jobs of J_1 are stored in the buffer, with the exception of job 1 which is processed upon release. Jobs 2 and 3 are then removed from the buffer and processed. Since 3-Partition has a solution, job 3 completes processing at time y . At this time the available buffer space is y . Job $3m + 1$ is released at time y and is processed immediately. Job $3m + 2$ is stored in the buffer during the interval $[y, y + 1]$, and then is processed in the interval $[y + 1, y + 2]$. All subsequent groups of three jobs from J_1 and two jobs from J_2 are processed similarly. Therefore, no jobs are lost in σ .

($\Leftarrow$) Consider first the interval $[0, y]$. Let $S_1 \subset J_1$ denote the set of jobs that complete processing by time y . By definition, $\sum_{j \in S_1} p_j \leq y$. Suppose that $\sum_{j \in S_1} p_j < y$. Then $\sum_{j \in S_1} s_j < y$ since $s_j = p_j$ for $j \in J_1$. Because $b_I = \sum_{j=1}^{3m} s_j$, the available space in the buffer when jobs $3m + 1$ and $3m + 2$ are released at time y is $\sum_{j \in S_1} s_j < y$. Consequently, one of these jobs is lost. Therefore, $\sum_{j \in S_1} p_j = y$, and only jobs of S_1 are processed in the interval $[0, y]$. Using a similar argument for the jobs scheduled between each successive release date $y, 2y + 2, \dots, my + 2(m - 1)$, a nonpreemptive schedule is obtained. Moreover, the subsets of J_1 that are processed between these release dates define a 3-Partition. $\square$

Corollary 3.2 The recognition version of problem $1|b_I, pmtn, r_j|\sum U_j$ is unary NP-complete.

Proof The result follows from theorem 3.3, since this problem is a generalization of $1|b_I, pmtn, r_j, d_j = \infty|\sum U_j$. $\square$

Finally, we discuss problems with unit storage requirements. Recall from theorem 3.1 that Algorithm SRPT solves problem $1|b_I, pmtn, r_j, d_j = \infty, s_j = 1|\sum U_j$ in $O(n \log n)$ time. However, problem $1|b_I, pmtn, r_j, s_j = 1|\sum U_j$ remains open with respect to both polynomial and pseudo-polynomial time solvability.

4. Weighted number of lost jobs

We first consider problems with a fixed buffer capacity. Then, we analyze problems with an arbitrary buffer capacity.

We present a pseudo-polynomial time dynamic programming algorithm to solve problem $1|b_I = b, pmtn, r_j|\sum w_j U_j$. As in Algorithm DPV, our algorithm generates (partial) schedules. Each schedule corresponds to some state (T, B, R) , where T is the completion time of the schedule and $T \in \mathcal{T}' = \mathcal{T} \cup \{r_1, \dots, r_n\}$, B is a vector that indicates which jobs are either in the buffer or on the machine at the end of the schedule (any jobs that are released at the end of the schedule and are not lost are included in B), and R is a vector of integers that specifies the remaining processing times for the jobs of B . Let $f(T, B, R)$ denote the maximum weight of completed jobs among all schedules that achieve the state (T, B, R) .

The state (T, B, R) is reached from some previous state $(\bar{T}, \bar{B}, \bar{R})$ by scheduling one unit of processing in the interval $[T-1, T]$ or scheduling idle time in the interval $[\bar{T}, T]$. If $T-1 \in \mathcal{T}$, then $\bar{T} = T-1$; otherwise, unavoidable idle time occurs in the interval $[\bar{T}, T]$, where $\bar{T} = \max\{t' | t' \in \mathcal{T}', t' < T\}$. State transitions are similar to those in Algorithm DPV. Also, we use B' , where $\{i | i \in N, B'_i = 1\}$ is the set of jobs that are in B and are released at time T .

The recurrence relation is

$$f(T, B, R) = \max \begin{cases} \max_{j \in H_1} \{w_j + f(T-1, B + e_j - B', R + e_j - \sum_{i=1}^n B'_i p_i e_i)\}, & (6) \\ \max_{j \in H_2} \{f(T-1, B - B', R + e_j - \sum_{i=1}^n B'_i p_i e_i)\}, & (7) \\ f(T', \mathbf{0}, \mathbf{0}), & \text{if } B = B' \text{ and } R = \sum_{i=1}^n B'_i p_i e_i, & (8) \end{cases}$$

for $(B, R) \in (\mathcal{B}, \mathcal{R})$, and $T \in \mathcal{T}'$,

where $T' = \max\{t' | t' \in \mathcal{T}', t' < T\}$, $B' = (B'_1, \dots, B'_n)$ with

$$B'_i = \begin{cases} 1 & \text{if } B_i = 1 \text{ and } r_i = T, \\ 0 & \text{otherwise,} \end{cases}$$

and where

$$H_1 = \{k | k \in N, R_k = 0, T' \leq d_k - 1, (B - B')S \leq b\},$$

$$H_2 = \{k | k \in N, R_k > 0, B'_k = 0, T' \leq d_k - R_k - 1, (B - e_k - B')S \leq b\}.$$

The boundary condition is

$$f(r_{\min}, B, \sum_{i=1}^n B_i p_i e_i) = 0 \text{ for all } B \in \mathcal{B} \\ \text{such that } \{i | i \in N, B_i = 1\} \subseteq \{i | i \in N, r_i = r_{\min}\}.$$

By implication, all other initial values are set to minus infinity.

The optimal value is given by

$$\max_{T \in \mathcal{T}'} \{f(T, \mathbf{0}, \mathbf{0})\}.$$

We refer to this dynamic programming approach as Algorithm DPT (where the T describes the first state variable).

Theorem 4.1 Algorithm DPT generates an optimal schedule for problem $1|b_I = b, pmtn, r_j|\sum w_j U_j$ in $O(n^{b+3} p_{\max}^{b+2})$ time.

Proof Equation (6) considers the situation in which job j completes processing at time T . In equation (7), job j is processed in the interval $[T-1, T]$, but does not complete. Equation (8) schedules idle time in the interval $[T', T]$ since the buffer is empty during this period. The choice of state variables $T \in \mathcal{T}'$ is justified by lemma 2.2. Since all possible ways of reaching state (T, B, R) are considered, it follows that Algorithm DPT generates an optimal schedule.

As in Algorithm DPV, $|\mathcal{B}, \mathcal{R}|$ is $O(n^{b+1} p_{\max}^{b+1})$. Lemma 2.2 and the definition of $\mathcal{T}'$ establish that $|\mathcal{T}'| \leq n(p_{\max} + 1)$. Consequently, the number of possible states (T, B, R) is $O(n^{b+2} p_{\max}^{b+2})$. Similar to DPV, each time the recurrence relation is used, $O(n)$ comparisons are required, each of which takes constant time. Thus, the time complexity of the recurrence relation is $O(n^{b+3} p_{\max}^{b+2})$. The computation of the optimal schedule from the final states requires $O(np_{\max})$ time. Thus, the overall time complexity of Algorithm DPT is $O(n^{b+3} p_{\max}^{b+2})$. $\square$

Corollary 4.1 Algorithm DPT generates an optimal schedule for problems $1|b_I = b, pmtn, r_j, d_j = \infty, s_j = 1|\sum w_j U_j$, $1|b_I = b, pmtn, r_j, d_j = \infty|\sum w_j U_j$ and $1|b_I = b, pmtn, r_j, s_j = 1|\sum w_j U_j$ in $O(n^{b+3} p_{\max}^{b+2})$ time.

Proof The result follows from theorem 4.1 since these problems are special cases of $1|b_I = b, pmtn, r_j|\sum w_j U_j$. $\square$

The next two results complete the complexity analysis of the problems considered in theorem 4.1 and corollary 4.1.

Theorem 4.2 The recognition version of problem $1|b_I = b, pmtn, r_j, d_j = \infty, s_j = 1|\sum w_j U_j$ is binary NP-complete.

Proof Nawijn et al. [8] prove that the recognition version of problem $1|b_I = b, r_j, d_j = \infty, s_j = 1|\sum w_j U_j$ is binary NP-complete. Examination of their constructed instance of that problem shows that the use of preemptions does not reduce the optimal cost. This establishes that the recognition version of problem $1|b_I = b, pmtn, r_j, d_j = \infty, s_j = 1|\sum w_j U_j$ is also binary NP-complete. $\square$

Corollary 4.2 The recognition versions of problems $1|b_I = b, pmtn, r_j, d_j = \infty|\sum w_j U_j$, $1|b_I = b, pmtn, r_j, s_j = 1|\sum w_j U_j$ and $1|b_I = b, pmtn, r_j|\sum w_j U_j$ are binary NP-complete.

Proof The results follow from theorem 4.2 since these problems are generalizations of $1|b_I = b, pmtn, r_j, d_j = \infty, s_j = 1|\sum w_j U_j$. $\square$

We now examine the complexity of problems with arbitrary buffer storage requirements.

Corollary 4.3 The recognition versions of problems $1|b_I, pmtn, r_j, d_j = \infty, s_j = 1|\sum w_j U_j$ and $1|b_I, pmtn, r_j, s_j = 1|\sum w_j U_j$ are binary NP-complete.

Proof Since these problems are generalizations of $1|b_I = b, pmtn, r_j, d_j = \infty, s_j = 1|\sum w_j U_j$, the result follows from theorem 4.2. $\square$

However, the complexity status of the two problems in corollary 4.3 remains open with respect to pseudo-polynomial time solvability.

Corollary 4.4 The recognition versions of problems $1|b_I, pmtn, r_j, d_j = \infty|\sum w_j U_j$ and $1|b_I, pmtn, r_j|\sum w_j U_j$ are unary NP-complete.

Proof The results follow from theorem 3.3 because these problems are generalizations of $1|b_I, pmtn, r_j, d_j = \infty|\sum U_j$. $\square$

5. Concluding remarks

This work provides algorithms and computational complexity results for several preemptive scheduling problems with finite capacity input buffers. Future research needs to focus on the resolution of the open preemptive scheduling problems described here. To make them more practical, it is important to improve the order of complexity of the pseudo-polynomial time dynamic programming algorithms. Also, lower bounds and computationally efficient procedures are needed for the NP-hard problems. Finally, the development of on-line procedures, for problems in which job data is not known until a job arrives, is a further research topic of interest.

Acknowledgements

This research was supported in part by the Summer Fellowship Program (Fisher College of Business, The Ohio State University) and a grant from The Royal Society.

References

- [1] M.R. Garey and D.S. Johnson, *Computers and Intractability: A Guide to the Theory of NP-Completeness*, Freeman, San Francisco, 1979.

- [2] R.L. Graham, E.L. Lawler, J.K. Lenstra and A.H.G. Rinnooy Kan, Optimization and approximation in deterministic machine scheduling: A survey, *Annals of Discrete Mathematic* 5(1979) 287–326.
- [3] N.G. Hall, M.E. Posner and C.N. Potts, Scheduling with finite capacity output buffers, *Operations Research* (1997), to appear.
- [4] N.G. Hall, M.E. Posner and C.N. Potts, Scheduling with finite capacity input buffers, *Operations Research* (1997), to appear.
- [5] R.L. Kruse, *Data Structures and Program Design*, 2nd ed., Prentice-Hall, Englewood Cliffs, NJ, 1987.
- [6] J.O. McClain and L.J. Thomas, *Operations Management*, 2nd ed., Prentice-Hall, Englewood Cliffs, N.J., 1985.
- [7] W.M. Nawijn, Minimum loss scheduling problems, *European Journal of Operational Research* 56 (1992)364–369.
- [8] W.M. Nawijn, W. Kern and S. Baas, Scheduling jobs with release times on a machine with finite storage, *European Journal of Operational Research* 74(1994)120–127.